

KUGGLE × MIRAE ASSET

LLM 활용과 AI 자동화의 기초

*ChatGPT를 넘어, 코드와 결합하는 LLM —
증권사 콘텐츠 파이프라인 사례와 함께*

TARGET

KUGGLE 13기 & 14기

RUNTIME

30분 (Q&A 포함)

FORMAT

강의 + 라이브 실습
(Colab)

SPEAKER

KUGGLE 12기 천시원

저는 이렇게 LLM을 씁니다

ME

M 증권사 · 상품컨설팅본부

ETF 콘텐츠 자동화 파이프라인 구축 및 운영
데이터 분석 지원 및 AX 업무 효율화

Track Record

- M증권사 상품솔루션팀 인턴 (DX, AX 업무 담당)
- LLM/RAG 기반 사내 자동화 빌드
- 리스크 관리 및 상품 마케팅 담당

어쩌다 LLM이었나

통계·ML로 데이터를 다뤘지만, 모델보다 "의사결정에 쓸 수 있는 한 줄 인사이트"가 늘 부족했음.
LLM은 그 마지막 1걸음을 메우는 도구.

지금 회사에서 만드는 것

ETF 트렌드 → ML 클러스터링 → LLM 분석 → HTML 리포트 4종 → 주 1회 이메일. 매주 수요일 9시
윈도우 작업 스케줄러가 자동 실행.

왜 여러분에게 이 애길 하나

ML/통계 백그라운드 + LLM API 가벼운 활용 → 실무 1년 차에 가장 빠르게 차이를 만드는 조합.

오늘 같이 볼 4가지

01

8'

LLM API 기초

ChatGPT 웹이 아닌 "호출 가능한 모델"
→ 코드에 LLM을 꽂는다는 의미



02

5'

바이브 코딩 (Vibe Coding)

Claude Code · Codex · Cursor
→ "코드 짜지 마라, 의도를 짜라"



03

9'

증권사 현업 파이프라인

ETF 트렌드 분석 파이프라인
ML × LLM 결합, RAG, 품질검증



04

8'

Colab 라이브 실습

5분이면 만드는 LLM 호출 →
클러스터 자동 네이밍 미니 데모



"끝나면 LLM API 호출 한 번은 직접 해보고 갑니다."

PART 01

LLM, ChatGPT 웹만 써봤다면 절반은 모르는 것

「웹 인터페이스」와 「API 호출」은 완전히 다른 사용 방식입니다.

왜 ChatGPT 웹이 아니라 "API" 인가

📄 웹 인터페이스 (ChatGPT.com)

Pros

- ✓ 누구나 즉시 시작
- ✓ 대화형 — 직관적

Cons

- ✗ 자동화 불가 (사람이 매번 타이핑)
- ✗ 코드/시스템과 연결 불가
- ✗ 모델·파라미터·출력 형식 통제 약함
- ✗ 비용 가시성·로그 관리 어려움

⚡ API 호출 (Anthropic / OpenAI SDK)

Pros

- ✓ 코드 한 줄로 LLM 호출 → 자동화 가능
- ✓ 함수처럼 다른 시스템에 끼워 넣기 OK
- ✓ Model / Temperature / JSON 출력 통제
- ✓ 로그·비용·캐시·재시도까지 코드로

Cons

- ✗ API Key 필요 (월 \$ 단위 과금)
- ✗ 기초 Python 한 줄은 알아야 함

💡 **핵심: LLM을 "채팅 상대"가 아니라 "함수"로 다루기 시작하는 순간 — 자동화 시스템화가 시작됩니다.**

LLM API, 이 4가지만 알면 시작합니다

01

Model

어떤 모델을 호출할지
claude-sonnet-4-5 / gpt-5
— 성능·속도·비용 trade-off

```
model="claude-sonnet-4-5"
```

02

Messages

대화 히스토리 배열
system(역할) + user(질문)
+ assistant(이전 답변)

```
[{"role": "user", "content": "..."}]
```

03

Temperature

출력의 무작위성 (0~1)
0 = 결정적, 분석용
0.7 = 창의적, 콘텐츠용

```
temperature=0.2
```

04

Max Tokens

응답 최대 길이 = 비용 상한
한글 1자 ≈ 약 1.5~2 토큰
출력 길이 = 비용

```
max_tokens=1024
```

👉 이 4가지가 LLM API 호출의 "손잡이". 더 깊은 옵션은 다 부수적입니다.

Claude API — 진짜 "5줄"이면 동작합니다

first_call.py

```
import anthropic

client = anthropic.Anthropic() # ANTHROPIC_API_KEY 환경변수 사용
msg = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=1024,
    messages=[{"role": "user", "content": "ETF 한 줄 설명"}],
)

print(msg.content[0].text)
```

🎯 단 5줄로 일어나는 일

- 1 SDK 로드**
anthropic 라이브러리만 import
- 2 Client 생성**
.env API Key 자동 인식
- 3 Messages 호출**
model + 질문 한 줄
- 4 응답 객체**
msg.content[0].text
- 5 다음 단계로**
이걸 함수로 감싸면 자동화 시작

📌 실습 슬라이드에서 Colab으로 직접 돌립니다.

프롬프트 엔지니어링 — 외울 건 4가지



역할 부여 (Role)

system 메시지에 "너는 미래에셋증권 ETF 애널리스트" — 모델의 페르소나 고정

```
system="너는 ETF 애널리스트다. 수치는 [확인 필요] 태그."
```



Few-shot 예시

원하는 출력 포맷 2~3개 예시를 함께 줌. 모델은 "패턴 매칭 기계"

```
예시1) 입력→출력 / 예시2) 입력→출력 / 본문) 입력→?
```



Structured Output

JSON 스키마 강제 → 후처리 자동화. Anthropic tools/OpenAI structured output

```
{"intent":"informational","confidence":0.92}
```



Chain-of-Thought

"단계별로 생각하고 답하라" — 추론형 작업의 정확도 ↑ (비용·지연도 ↑)

```
"먼저 단계별 분석 → 최종 결론 한 줄"
```

⚠ 프롬프트는 "한 번에 잘 짜는 것"이 아니라 "실패 → 수정" 사이클로 진화시키는 코드입니다.

PART 02

바이브 코딩 (Vibe Coding) — 코드 짜지 마라, 의도를 짜라

"한 줄 한 줄 타이핑"에서 "무엇을 왜 만들지 명확히 적기"로 추상화가 한 단 올라간 코딩 방식.

지금 쓰는 3대 도구



Claude Code

Anthropic · CLI/IDE Agent

터미널/IDE에서 파일 읽고·수정·테스트까지 실행하는 "코딩 에이전트". 큰 변경·리팩토링·신규 기능 빌드에 강함.

 복잡한 멀티파일 작업



Codex (OpenAI)

ChatGPT · Web/CLI Agent

ChatGPT 안의 에이전트 + 자체 CLI. 클라우드 가상머신에서 코드 실행·PR 생성. 단발 작업·스크립트 빠르게.

 스크립트·자동화 단위



Cursor

VS Code 기반 IDE

에디터 안에서 인라인 채팅 + Tab 자동완성. 한 줄 한 줄 보면서 작업하는 스타일에 적합. 학습 곡선 가장 낮음.

 처음 시작하는 사람

 멘토 입장에서선: "하나 골라 깊게 — 작은 자동화 한 개 끝까지 만들어 보기."

바이브 코딩, 안 망하는 실전 5가지

1

Context > Prompt

프롬프트보다 "어떤 파일·어떤 제약·어떤 결과물"을 같이 주는 게 압도적으로 중요.

2

작은 단위로 자르기

"앱 만들어줘" ❌ / "이 함수의 입력 검증만 추가" ✅

3

스스로 검증 시키기

테스트 작성 → 실행 → 실패면 자동 수정. "끝났다"는 모델의 말 ≠ 동작

4

PR 단위로 끊기

한 작업 = 한 커밋/한 브랜치. 망가지면 즉시 롤백 가능한 안전망.

5

사람이 마지막 review

비즈니스 로직·보안·금융 수치는 무조건 사람이 마지막 눈으로 확인.

🗨️ "AI가 짠 코드" 보다 "AI에게 어떤 의도를 줬는지"가 본인의 실력으로 남습니다.

PART 03 · CASE STUDY

증권사 현업에서 돌고 있는 ETF 트렌드 분석 파이프라인

미래에셋증권 상품솔루션팀 · 매주 수요일 09:00 자동 실행 · 11 단계 · LLM 비용 약 \$1/주

11단계 파이프라인을 4블록으로 보면

① 수집 (Ingestion)

Step 1·2·3·5·6·7

- 1 네이버 블로그 API
- 2 네이버 뉴스 API
- 3 DataLab 검색량
- 4 YouTube 영상

② ML 분석

Step 1 / 5 내부

- 1 TF-IDF 키워드
- 2 K-Means 클러스터
- 3 LDA 토픽
- 4 주차별 누적 DB

③ LLM 콘텐츠 생성

Step 10

- 1 S1 Search Intent
- 2 S2 JTBD 3레이어
- 3 S3 경쟁사+RAG
- 4 Content Ideas·Brief

④ 산출·배포

Step 4·8·9·11

- 1 블로그 보고서 HTML
- 2 뉴스 보고서 HTML
- 3 콘텐츠 브리프 HTML
- 4 주간 이메일 발송

 매주 수요일 09:00 Windows 작업 스케줄러가 run_all.py 한 번 호출 → 11 단계 순차 실행

여기가 핵심 — ML과 LLM은 "역할 분담"

ML이 잘하는 것

▶ 수치적·확정적 작업

TF-IDF 가중치 / 코사인 유사도 / Silhouette 점수

▶ 대량 처리·재현 가능

수천 문서를 결정적·반복적으로 클러스터링

▶ 근거 있는 "숫자"

키워드 빈도, 신규성, 클러스터 크기 — 객관 수치

▶ But 해석은 못한다

"cluster_3 = {달러, 비트코인, 기관}" 까지가 한계

LLM이 잘하는 것

▶ 맥락·언어·해석

ML 클러스터 → "레버리지/매수/사람" 같은 의미 있는 네이밍

▶ 구조화된 추론

JTBD 3레이어, Search Intent 분류, AIDA/PAS 프레임

▶ 비정형 → 정형 변환

뉴스·블로그 본문 → JSON {topic, sentiment, claim}

▶ But 수치 환각 위험

[확인 필요] 태그 강제 + Quality Check 단계 필수

 ML은 "무엇이 묶이는가", LLM은 "왜 묶이고 뭐라 부를까" — 둘이 만났을 때 비로소 보고서가 됩니다.

Step 10 LLM 단계 — 5 stage + 품질 검증

S1	S2	S3	S4	S5
Search Intent <i>Sonnet</i>	JTBD <i>Opus → Sonnet fallback</i>	Competitor + RAG <i>Gemini Search → Opus</i>	Content Ideas <i>Sonnet</i>	Trend Brief <i>Gemini + Opus</i>
Informational / Commercial / Transactional 분류	투자자 Job-To-Be-Done 3레이어 (functional/emotional/social)	Gemini Search Grounding으로 실시간 경쟁사 분석	카드뉴스 3 / 블로그 5 / SNS 5 / 유튜브 — AIDA·PAS 프레임	Top5 트렌드 시그널 + 콘텐츠 방향

🛡️ Quality Check (할루시네이션 검증) — 별도 LLM 호출

▶ 수치 단정 탐지

"+42.7%" 같은 수치에 출처 없으면 플래그

▶ [확인 필요] 강제

데이터 의존 주장은 태그 필수

▶ AIDA/PAS 점수

프레임 준수율 10점 만점

▶ 할루시 리포팅

최종 HTML 상단에 빨간 박스로 노출

💰 S3 Gemini 캐시·Opus→Sonnet 폴백으로 비용 ~\$0.99/주 · 단위 테스트 81개 통과

RAG·LLM Wiki — 모델 "밖"의 지식을 끌어오는 법

RAG (Retrieval-Augmented Generation)

외부 문서를 "검색해서" LLM 프롬프트에 끼워주는 패턴.
→ 모델 학습일 이후 정보, 사내 데이터, 정확한 출처를 다룰 때 필수.

1. **1. Query** 사용자 질문
2. **2. Retrieve** 벡터DB·검색API에서 관련 문서 K개
3. **3. Augment** 프롬프트에 문서 본문 삽입
4. **4. Generate** LLM이 출처 기반 응답 생성

우리 파이프라인에선 이렇게 씁니다

Gemini Search Grounding

S3 단계에서 "이번 주 경쟁사 ETF 콘텐츠"를 실시간 검색 → 결과를 LLM 컨텍스트로.

사내 RAG (TIGER 상품DB)

TIGER 종목 코드·운용 방식을 LLM이 인용. 환각 방지 + 출처 일관성.

LLM Wiki = 사내 지식 인덱스

사내 문서·리포트를 임베딩 → 벡터DB. "이 주제 우리 보고서 어디?" 즉시 답.

캐시 = 비용 최적화

S3 결과를 JSON으로 캐시 — 같은 클러스터 재실행 시 비용 절반.

 **통계과 입장 한 줄: 벡터 임베딩·유사도 검색·평가는 결국 모두 "통계와 거리 측정"입니다.**

매주 수요일 09:00, 실제로 나오는 결과물



Step 4

블로그 트렌드 리포트

네이버 블로그 ETF 키워드 → TF-IDF / K-Means / LDA → HTML 리포트



Step 9

블로그 × 뉴스 교차 분석

공통 / 블로그 / 뉴스 클러스터 매핑
— 우선 제작 주제 식별

Step 10

LLM 트렌드 브리프

Top 5 시그널 + 콘텐츠 방향 + AIDA/PAS 품질 점수 + 할루시네이션 박스



Step 10

LLM 콘텐츠 아이디어

클러스터별 카드뉴스 3 / 블로그 5 / 유튜브 3 / SNS 5 — 마케팅 즉시 사용

Colab 실습 — 5분 컷 미니 자동화

STEP 1 — 셋업

```
!pip install anthropic

import os, anthropic
os.environ["ANTHROPIC_API_KEY"] = \

"sk-ant-..."

client = anthropic.Anthropic()

# Console에서 발급 →
# console.anthropic.com
```

STEP 2 — 한 번 호출

```
def ask(q):
    r = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=512,
        system="한 줄로 답하라",
        messages=[{
            "role": "user",
            "content": q
        }])
    return r.content[0].text

print(ask("ETF란?"))
```

STEP 3 — 자동 네이밍

```
# 미니 ML → LLM 결합
clusters = {
    "c1": ["비트코인", "달러", "기관"],
    "c2": ["레버리지", "매수", "손실"],
}

for k, kws in clusters.items():
    q = f"{kws} 한 단어로 네이밍"

    print(k, ask(q))
```

 실행 결과 예: c1 → "디지털자산 기관유입" · c2 → "레버리지 매수심리" (수업에서 라이브로 돌립니다)

오늘 가져갈 5가지

1

LLM은 "채팅" 아닌 "함수"다

API로 호출하는 순간 자동화·시스템화의 문이 열린다.

2

프롬프트 4무기로 충분히 시작

Role · Few-shot · JSON · CoT — 외울 건 이 4가지.

3

바이브 코딩은 의도 설계

코드 줄 수보다 "무엇을·왜·어떤 제약"이 본인 실력.

4

ML × LLM = 진짜 보고서

ML로 묶고, LLM으로 해석·이름 짓고, Quality Check로 환각 잡기.

5

작은 자동화 1개부터

주간 리포트·뉴스 요약·강의노트 정리 — 자기 use case 1개를 끝까지.



Q & A — 진로·도구·실무·논문 다 환영합니다. (편하게 질문 주세요)